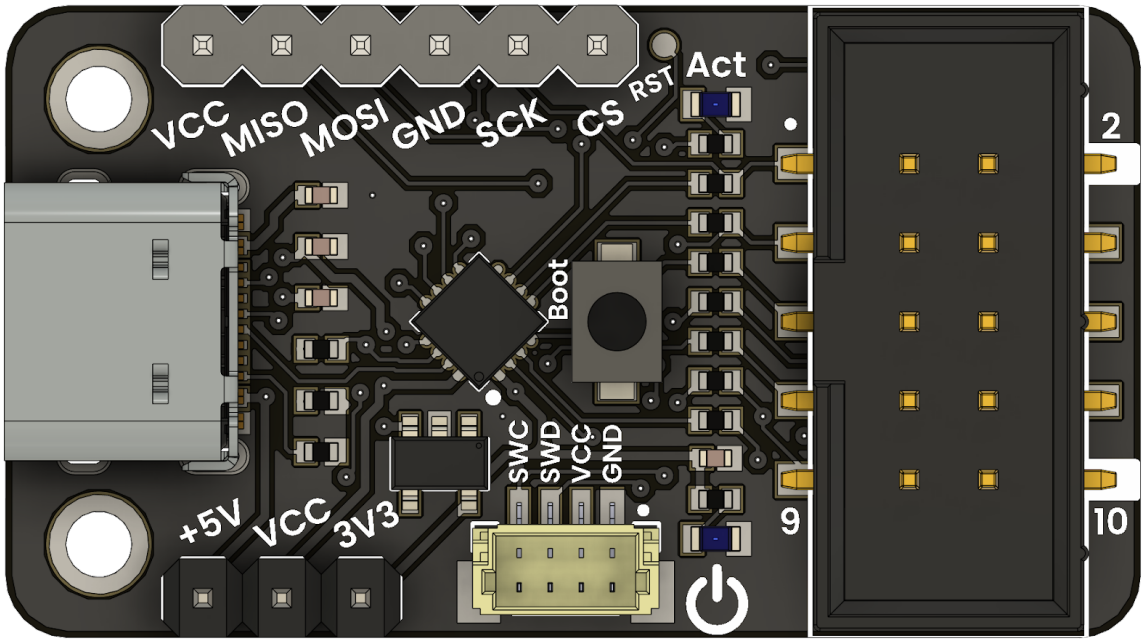


CH552 USB Multi-Protocol Programmer User Guide and Technical Reference

Release 0.0.1

Department of Research, Innovation, and Development

Jun 27, 2026

Contents

Note: This documentation is actively evolving. For the latest updates and revisions, please visit the project's GitHub repository.

Multi-Protocol Programmer

The **USB Multi-Protocol Programmer** is a compact and cost-effective device designed for embedded systems development, testing, and debugging. It supports multiple hardware architectures including **AVR**, **ARM Cortex-M (CMSIS-DAP)**, and **CPLD (MAX II)**, making it ideal for a wide range of applications such as firmware development, educational labs, and low-volume production environments.

This programmer is built around the **CH552 microcontroller**, which is based on the enhanced **8051 architecture**. It offers native USB support and a range of digital interfaces (GPIO, SPI, I2C, UART), enabling seamless communication between the host system and the target hardware.

Microcontroller Core

The programmer integrates a **CH552 microcontroller** with the following characteristics:

- 8051-based enhanced core, up to 24 MHz.
- Native USB 2.0 Full-Speed device.
- Multiple GPIO pins for signal control and mapping.
- SPI, I2C, and UART interfaces for protocol bridging.
- Low power consumption and small form factor.

Features

- **Multi-architecture support**: Compatible with AVR (ISP), ARM Cortex-M (CMSIS-DAP), and CPLD (JTAG).
- **In-System Programming (ISP)**: Flash microcontrollers without desoldering.
- **Real-time debugging**: Step-through and breakpoint debugging with OpenOCD and PyOCD.
- **JTAG boundary-scan**: For CPLD configuration and board testing.
- **Configurable GPIOs**: Adaptable for use as JTAG, SWD, or ISP lines.
- **USB 2.0 interface**: Direct connection to host PC using USB CDC or HID.
- **Toolchain compatibility**: Works with avrdude, OpenOCD, PyOCD, urJTAG, and others.
- **Cross-platform support**: Compatible with Linux and partially supported on Windows.

Advantages

- **Compact design**: Suitable for breadboards and embedded setups.
- **Versatility**: One device for multiple programming and debugging protocols.
- **Open-source firmware**: Fully customizable and community-supported.
- **Cost-effective**: Inexpensive alternative to commercial debuggers and programmers.
- **Linux-friendly**: No need for proprietary drivers on Linux systems.
- **Ideal for education**: Can be used in microcontroller courses and workshops.

Limitations

- **External power required**: Cannot supply power to high-current target boards.
- **Learning curve**: Requires knowledge of protocols like CMSIS-DAP, JTAG, or AVR ISP.
- **Firmware updates**: May require reflashing to support new features or targets.
- **Partial Windows support**: Some tools may require manual setup or driver adjustments.

Compatibility

CMSIS-DAP (ARM Cortex-M)

- Compatible with CMSIS-DAP v2.0 protocol.
- Supported by OpenOCD and PyOCD.
- Tested with:
  - STM32F0
  - RP2040 (Raspberry Pi Pico)
  - PY32 series
  - Other Cortex-M0/M3/M4 devices

AVR ISP

- Works with avrdude using USBasp-like interface.
- Supports:
  - ATmega328P
  - ATtiny85
  - ATmega2560
  - Other classic 8-bit AVR microcontrollers

CPLD JTAG

- Supports Intel (formerly Altera) MAX II series.
- Compatible with JTAG tools like urJTAG or openFPGALoader.
- JTAG signals exposed via GPIO (TDI, TDO, TCK, TMS).

Use Cases

- Firmware flashing and in-system programming.
- Debugging embedded applications with CMSIS-DAP.
- Educational labs and training environments.
- Low-cost production line programming.
- Boundary-scan tests for hardware bring-up.
- CPLD configuration and prototyping.

Resources

- Firmware: [https://github.com/wagiminator/CH552-DAPLink{}](https://github.com/wagiminator/CH552-DAPLink)
- CH552 Datasheet: Available from WCH official website.
- Tools:
  - OpenOCD, PyOCD
  - avrdude
- Community support: GitHub issues, Reddit, Hackaday, forums.

CHAPTER
ONE

TERMS, ACKNOWLEDGMENTS, AND LICENSES

1.1 Terms and Conditions

By using, modifying, or distributing the documentation, firmware, or hardware designs included in this repository, you agree to the following terms:

- All materials are provided **"as-is"**, without warranty of any kind.
- The authors and contributors shall not be held responsible for **any damages**, data loss, or legal issues arising from the use of these materials.
- Usage is intended for **educational, development, prototyping**, and other lawful purposes.
- When redistributing or reusing any part of this project, you must **retain attribution** and comply with the corresponding license terms of each component.

1.2 Acknowledgments and Contributors

This project builds upon the work of several open-source developers and projects:

1.2.1 CMSIS-DAP (DAPLink Firmware for CH552)

- **Stefan Wagner** Project: CH552-DAPLink License: Creative Commons BY-SA 3.0 Description: CMSIS-DAP firmware and hardware design
- **Ralph Doncaster** Source: nerdralph/ch554_sdcc Description: Original CMSIS-DAP firmware implementation for CH554 (SDCC)
- **Deqing Sun** Source: CH55xduino Description: CH552/CH554 Arduino-compatible toolchain

1.2.2 USB-Blaster Firmware (CH552G)

- **Vladimir Duan** Project: CH55x-USB-Blaster License: MIT Description: USB-Blaster JTAG emulation for CH55x
- **Blinkinlabs** SDK Source: ch554_sdcc Description: SDK for CH552/CH554 (SDCC)
- **Doug Brown** Blog: Fixing a Knockoff Altera USB Blaster Description: Insights into compatibility and firmware flashing

1.3 Hardware License

All hardware designs (schematics, layouts, and design files) in this repository are released under the **MIT License**, allowing unrestricted use, modification, and distribution, provided the original license and attribution are retained.

1.4 Resources and References

Table 1.1: Source URLs

Project / Tool	Source URL
CH552 DAPLink	https://github.com/wagiminator/CH552-DAPLink
picoDAP	https://github.com/wagiminator/CH552-picoDAP
CH55xDuino	https://github.com/DeqingSun/ch55xduino
CMSIS-DAP Handbook	https://os.mbed.com/handbook/CMSIS-DAP
CH55x USB-Blaster	https://github.com/VladimirDuan/CH55x-USB-Blaster
SDCC Compiler	https://sdcc.sourceforge.net/
CH554 SDK	https://github.com/Blinkinlabs/ch554_sdcc

1.5 Licenses

1.5.1 Documentation & Visual Content

This user guide and its visual content are licensed under:

Creative Commons Attribution-ShareAlike 3.0 Unported License

1.5.2 Firmware Projects

- **CH552-DAPLink**: Creative Commons BY-SA 3.0 — © Stefan Wagner
- **CH55x-USB-Blaster**: MIT License — © Vladimir Duan
- **CH55x SDK / Tools**: MIT License — © Blinkinlabs

1.5.3 Hardware Repository

- All PCB designs and schematics are released under the **MIT License**.

Note: If you distribute this product with third-party firmware (e.g., CMSIS-DAP), you are responsible for ensuring license compliance. Only firmware developed by Unit Electronics and released under the MIT license is supported for commercial redistribution.

1.5.4 Preloaded USB-Serial Firmware

This product may include preloaded firmware based on the project by **Kongou Hikari**: "USB to Serial Converter firmware for CH552T". Original source: *[https://github.com/diodep/ch55x_dualserial/tree/master]* License: MIT

Under the terms of the MIT License, users are free to modify or replace the firmware. Unit Electronics provides this firmware for convenience only and does not offer performance guarantees.

CHAPTER
TWO

UNIT CH55X SDK

The UNIT CH55x SDK is the CH55x firmware development base used by this documentation. The recommended implementation workflow is PlatformIO. PlatformIO uses SDCC and integrates the default upload flow validated on Linux and Windows.

Repository: UNIT-Electronics-MX/unit_ch55x_sdk

2.1 Recommended Workflow

Use the dedicated *PlatformIO Support for CH55x* chapter for installation, project configuration, build, upload, and supported boards.

The container-based Makefile workflow is optional and low priority. It is documented only in *Optional Docker and spkg* for legacy examples or isolated builds.

2.2 Features

- PlatformIO platform support for CH55x projects on Linux and Windows.
- SDCC-based firmware builds for `firmware.ihx`, `firmware.hex`, and `firmware.bin`.
- Default upload flow with `chprog.py` on Linux and `vnproch55x` on Windows.
- Board definitions for CH551, CH552, CH554, and CH559 targets.
- Optional container tooling for legacy Makefile projects.

2.3 Requirements

For the recommended PlatformIO workflow:

- Visual Studio Code with the PlatformIO IDE extension.
- Git.
- Python 3 on Linux.
- WCH USB driver CH372DRV on Windows for USB upload.

PlatformIO Core can also be used for command-line automation, but the recommended installation path is the VS Code extension.

2.4 macOS Status

macOS has not been tested in the current SDK validation process. Do not present macOS as a supported installation, build, or upload target until it has a verified test procedure.

2.5 Version Compatibility

Use SDK `v0.1.4` or newer for the current PlatformIO upload flow. This version avoids treating the Windows-only Arduino tools archive as a PlatformIO package on Linux and keeps the default `chprog.py` uploader path clean.

Projects can pin a released SDK version in `platformio.ini`:

```
[env:unit_ch552]
platform = https://github.com/UNIT-
↪Electronics-MX/unit_ch55x_sdk.git#v0.1.4
board = unit_ch552
```

2.6 Project Structure

```
unit_ch55x_sdk/
|-- boards/                 # PlatformIO␣
↪board definitions
|-- builder/                # PlatformIO␣
↪build scripts
|-- examples/               # CH55x␣
↪example projects
|-- platform.json           # PlatformIO␣
↪platform metadata
`-- README.md
```

2.7 License

This SDK is released under the MIT License.

Some example projects inside `examples/` are derived from the work of Stefan Wagner and are licensed under Creative Commons Attribution-ShareAlike 3.0 Unported (CC BY-SA 3.0). Each source file includes its origin and license header where applicable.

CHAPTER
THREE

PLATFORMIO SUPPORT FOR CH55X

PlatformIO is the recommended workflow for new CH55x projects. The `unit_ch55x_sdk` repository can be used as a local PlatformIO development platform or referenced directly from GitHub.

PlatformIO uses SDCC to build CH55x firmware and generates the `firmware.ihx`, `firmware.hex`, and `firmware.bin` output files.

Repository: UNIT-Electronics-MX/unit_ch55x_sdk

Note: Docker and manual SDCC/Make workflows are optional and low priority. For new Ubuntu and Windows setups, install PlatformIO, configure `platformio.ini`, and build with `pio run`. macOS has not been tested in the current validation process.

3.1 Requirements

- Visual Studio Code.
- PlatformIO IDE extension for VS Code.
- Git.
- Python 3 on Linux for the default uploader.
- WCH USB driver CH372DRV on Windows for USB bootloader uploads.

PlatformIO Core is optional and useful for terminal-based builds or CI, but the recommended installation method is the VS Code extension.

3.2 Install PlatformIO

Install the official PlatformIO IDE extension from the VS Code Extensions view:

1. Open the **Extensions** view in Visual Studio Code.
2. Search for `PlatformIO`.
3. Click **Install** on the official **PlatformIO IDE** extension.

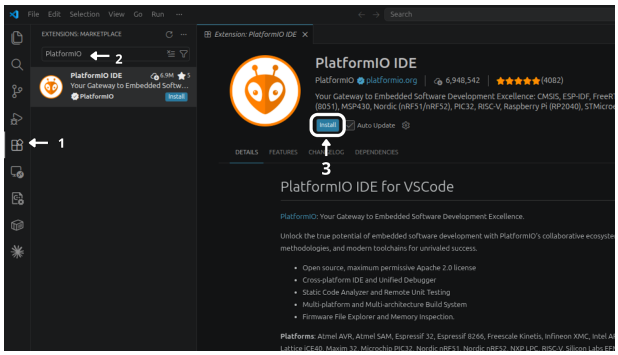


Fig. 3.1: PlatformIO IDE extension installation steps in Visual Studio Code.

After installation, the extension page shows PlatformIO IDE as installed and enabled in the current workspace:

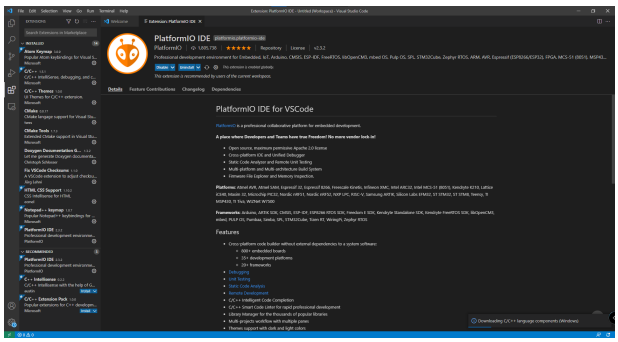


Fig. 3.2: PlatformIO IDE extension installed and enabled in Visual Studio Code.

3.2.1 Ubuntu / Linux

Install Visual Studio Code and then install the official PlatformIO IDE extension from the VS Code Extensions view. After the extension finishes its setup, restart VS Code and verify that the `pio` command is available:

```
pio --version
```

Install PyUSB in the PlatformIO Python environment if upload support needs it:

```
~/.platformio/penv/bin/python -m pip␣
↪install pyusb
```

USB access may also require a udev rule:

```
echo 'SUBSYSTEM=="usb", ATTR{idVendor}==
↪"4348", ATTR{idProduct}=="55e0", MODE=
↪"666"' | sudo tee /etc/udev/rules.d/99-
↪ch55x.rules
sudo udevadm control --reload-rules
sudo udevadm trigger
```

3.2.2 macOS

macOS has not been tested in the current SDK validation process. No macOS installation, build, or upload procedure is provided until the workflow is validated.

3.2.3 Windows

Install Visual Studio Code, the official PlatformIO IDE extension, Git, and the CH372 driver. The manual SDCC, MinGW, and Make setup is not required for PlatformIO projects.

Verify PlatformIO from PowerShell:

```
C:\Users\<user>\.platformio\penv\Scripts\
↪platformio.exe --version
```

3.3 Clone the SDK

For local development, clone the SDK:

```
git clone https://github.com/UNIT-
↪Electronics-MX/unit_ch55x_sdk.git
cd unit_ch55x_sdk
```

3.4 Build a PlatformIO Example

From the Blink example inside the SDK repository, run:

```
cd examples/platformio-blink
pio run
```

The generated firmware files are placed in:

```
examples/platformio-blink/.pio/build/unit_
↪ch552/
```

Other examples that include a `platformio.ini` file can be built the same way:

```
cd examples/usb/usb_uart
pio run
```

3.5 Upload Firmware

Put the CH55x board in bootloader mode and run:

```
pio run -t upload
```

The default `upload_protocol = auto` selects `chprog.py` on Linux and `vnproch55x` on Windows. macOS upload has not been validated.

3.5.1 Windows Upload with vnproch55x

Windows upload uses `vnproch55x.exe` from the `devlabtools` package. If PlatformIO does not download the uploader automatically, install it manually in the SDK root:

```
cd C:\path\to\unit_ch55x_sdk
Invoke-WebRequest "https://github.com/UNIT-
↪Electronics/Uelectronics-CH552-Arduino-
↪Package/releases/download/v0.0.6/
↪ch55xduino-tools_mingw32-2026.06.21.tar.
↪bz2" -OutFile "ch55xduino-tools_mingw32-
↪2026.06.21.tar.bz2"
tar -xjf ch55xduino-tools_mingw32-2026.06.
↪21.tar.bz2
Test-Path .\tools\win\vnproch55x.exe
```

The `Test-Path` command must print `True`. The extracted layout must contain:

```
tools/win/vnproch55x.exe
tools/win/*.dll
```

After installing the uploader, build and upload from the PlatformIO project:

```
C:\Users\<user>\.platformio\penv\Scripts\
↪platformio.exe run
C:\Users\<user>\.platformio\penv\Scripts\
↪platformio.exe run --target upload
```

3.6 PlatformIO Configuration

For a PlatformIO project inside this repository, point `platform` to the SDK root:

```
[env:unit_ch552]
platform = ../..
board = unit_ch552
```

To use a released SDK version from GitHub, pin a tag in `platformio.ini`:

```
[env:unit_ch552]
platform = https://github.com/UNIT-
↪Electronics-MX/unit_ch55x_sdk.git#v0.1.4
board = unit_ch552
```

Use SDK `v0.1.4` or newer for the current upload flow. Older checkouts may try to install Windows-only uploader archives as PlatformIO packages on Linux.

To use the current SDK branch directly from GitHub:

```
[env:unit_ch552]
platform = https://github.com/UNIT-
↪Electronics-MX/unit_ch55x_sdk.git
board = unit_ch552
```

For an existing Makefile-style example where `main.c` is located at the project root, set the source directory globally:

```
[platformio]
src_dir = .

[env:unit_ch552]
platform = ../..
board = unit_ch552
```

The default configuration can usually stay minimal:

```
[env:unit_ch552]
platform = ../..
board = unit_ch552

; Arduino defaults to bootcfg 3 for CH552␣
↪P3.6 (D+) pull-up.
board_upload.bootcfg = 3
```

You can force a specific uploader when needed:

```
upload_protocol = chprog
upload_protocol = vnproch55x
```

For serial upload:

```
upload_protocol = vnproch55x_serial
upload_port = COM5
```

3.7 Supported Boards

Board ID	Target
unit_ch551	CH551
unit_ch552	CH552
unit_ch554	CH554
unit_ch559	CH559

Common build overrides can be added to `platformio.ini`:

```
board_build.f_cpu = 24000000
build_flags =
  -D PIN_LED=P34
```

3.8 Create a PlatformIO Project

Create a project directory with a `platformio.ini` file and a source file under `src/`:

```
my_ch55x_project/
|-- platformio.ini
`-- src/
    `-- main.c
```

Use this minimal `platformio.ini`:

```
[env:unit_ch552]
platform = https://github.com/UNIT-
↪Electronics-MX/unit_ch55x_sdk.git#v0.1.4
board = unit_ch552
```

Build the project with:

```
pio run
```

Upload the project with:

```
pio run -t upload
```

CHAPTER
FOUR

OPTIONAL DOCKER AND SPKG

The UNIT CH55x SDK includes optional Docker-based tooling through `spkg` for Makefile-based examples. Use PlatformIO first for new projects; Docker is kept as a secondary workflow for isolated builds and legacy examples.

Use this page only when a project specifically needs the Docker/`spkg` workflow.

Note: This is the only chapter that documents `spkg` usage. For new project implementation, use *PlatformIO Support for CH55x*.

4.1 Requirements

4.1.1 Common Requirements

- Docker Desktop or Docker Engine.
- Git.
- Bash shell.

4.1.2 Linux

- Python 3.
- Permission to run Docker.
- Superuser privileges may be required during Docker installation or when the current user is not part of the `docker` group.

4.1.3 Windows

- Docker Desktop for Windows.
- Git Bash.
- Docker Desktop with WSL2 or Hyper-V backend enabled.
- MinGW64, included with Git Bash, for the `make` command.

4.2 Installation

Clone the SDK repository:

```
git clone https://github.com/UNIT-
↪Electronics-MX/unit_ch55x_sdk.git
cd unit_ch55x_sdk
```

On Linux, make the launcher executable:

```
chmod +x spkg/spkg
```

Optional global installation on Linux:

```
cd spkg
sudo ln -s "$(pwd)/spkg" /usr/local/bin/
↪spkg
```

On Windows, run the launcher from Git Bash:

```
./spkg/spkg.bat --help
```

4.3 Build the Docker Image

Start Docker Desktop or the Docker daemon before running the SDK commands.

Linux

```
./spkg/spkg compose
```

If `spkg` was installed globally:

```
spkg compose
```

Windows

```
./spkg/spkg.bat compose
```

Verify that Docker is running with:

```
docker ps
```

4.4 Compile a Project

Linux

```
./spkg/spkg -p ./examples/Blink
```

With global installation:

```
spkg -p ./examples/Blink
```

Windows

```
./spkg/spkg.bat -p ./examples/Blink
```

4.5 Run Make Targets

The command after the project path is forwarded to `make` inside the container. Common targets are `clean`, `all`, and `hex`.

Linux

```
./spkg/spkg -p ./examples/Blink clean
./spkg/spkg -p ./examples/Blink all
./spkg/spkg -p ./examples/Blink hex
```

Windows

```
./spkg/spkg.bat -p ./examples/Blink clean
./spkg/spkg.bat -p ./examples/Blink all
./spkg/spkg.bat -p ./examples/Blink hex
```

4.6 Create a New Project

The `init` command creates a new project directory.

Linux

```
./spkg/spkg init examples/project
```

Windows

```
./spkg/spkg.bat init examples/project
```

4.7 Output

For Makefile-based projects, the compiled binary is generated at:

```
examples/Blink/build/main.bin
```

Other generated files, such as HEX output, are written under the same project `build/` directory according to the project Makefile.

4.8 Flash Tools

Generated firmware can be flashed with one of the supported CH55x tools:

- `tools/chprog.py`.
- wchusbdfu.
- WCHISPTool.

4.9 Configure Docker Without sudo on Linux

Docker commands may require `sudo` until the current user is added to the `docker` group.

4.9.1 Install Docker Engine

Select one installation method.

docker.io package

```
sudo apt update
sudo apt install -y docker.io
```

Official Docker packages

```
sudo apt remove docker docker-engine␣
↪docker.io containerd runc
sudo apt update
sudo apt install -y ca-certificates curl␣
↪gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/
↪linux/ubuntu/gpg | sudo gpg --dearmor -o␣
↪/etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-
↪architecture) signed-by=/etc/apt/
```

(continues on next page)

(continued from previous page)

```
↪keyrings/docker.gpg] https://download.
↪docker.com/linux/ubuntu $(lsb_release -
↪cs) stable" | sudo tee /etc/apt/sources.
↪list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-
↪cli containerd.io docker-compose-plugin
```

4.9.2 Verify Docker Operation

```
sudo systemctl start docker
sudo systemctl enable docker
docker version
```

4.9.3 Add the User to the docker Group

```
sudo usermod -aG docker $USER
newgrp docker
```

Log out and back in if the group change is not applied in the current shell.

4.9.4 Validate Non-Privileged Docker Usage

Run Docker without `sudo`:

```
docker ps
```

The command should return either an empty container list or the column headers.

4.9.5 Verify Docker Compose

Check the modern Docker Compose plugin first:

```
docker compose version
```

Some systems also provide the classic command:

```
docker-compose version
```

If Docker Compose is missing, install the package for your distribution. On Ubuntu-based systems:

```
sudo apt install docker-compose-plugin
```

4.9.6 Optional Socket Permission Check

Inspect the Docker socket:

```
ls -l /var/run/docker.sock
```

Expected group ownership:

```
srw-rw---- 1 root docker ...
```

If the group or permissions are wrong:

```
sudo chown root:docker /var/run/docker.sock
sudo chmod 660 /var/run/docker.sock
```

4.10 Troubleshooting

- If `spkg compose` fails, confirm Docker is running with `docker ps`.
- If Linux requires `sudo` for every Docker command, re-check the `docker` group membership and start a new login session.
- If Windows cannot run `spkg`, use Git Bash and call `./spkg/spkg.bat`.
- If a build cannot find a Makefile target, verify that the path passed with `-p` points to the project directory that contains the Makefile.

CHAPTER
FIVE

GENERAL INFORMATION

The **Multi-Protocol Programmer** is a compact and versatile development tool designed for high-precision embedded system applications. It supports a broad range of protocols and device architectures, including **AVR**, **ARM (CMSIS-DAP)**, and **CPLD (MAX II)**. Its USB connectivity enables direct interfacing with standard development environments, enabling:

- In-system programming (ISP)
- Step-through debugging
- Boundary-scan testing (JTAG)
- Flash memory operations

5.1 Supported Architectures

- **AVR** — via ISP (SPI configuration)
- **ARM Cortex-M** — via CMSIS-DAP and SWD
  - **RP2040**
  - **PY32**
  - **STM32**
- **CPLD/FPGA (MAX II)** — via JTAG

All protocols are exposed via labeled headers or JST connectors, allowing fast, solderless prototyping.

This device connects to a host system via USB and allows the user to program and debug various microcontrollers and programmable logic devices.

5.2 Supported Interfaces

- **JTAG**, for full-chip debugging and boundary scan
- **SWD**, for ARM Cortex-M series
- **SPI**, for flash and peripheral programming
- **UART**, for serial bootloaders and communication
- **GPIO**, for bit-banging or peripheral testing

Table 5.1: Interface and Signal Overview

Interface	Description	Signals / Pins	Typical Use
JTAG	Standard boundary-scan and debug interface	TCK, TMS, TDI, TDO, nTRST	Full chip programming, in-circuit test, debug
SPI	High-speed serial peripheral interface	MOSI, MISO, SCK, CS	Flash memory programming, peripheral data exchange
SWD	ARM's two-wire serial debug and programming interface	SWCLK, SWDIO	Cortex-M programming and step-through debugging
JST Header	Compact connector for power and single-wire debug signals	SWC (SWCLK), SWD (SWDIO), VCC, GND	Quick-connect to target board for SWD and power

5.3 Sections GPIO Pin Distribution

The Multi-Protocol Programmer features a set of GPIO pins that can be configured for various protocols, including JTAG, SWD, and ISP. These GPIOs are mapped to specific functions in the firmware, allowing users to adapt the programmer for different applications.

The GPIO pin distribution is defined within the CH552 firmware, supporting flexible assignment for various protocols. The firmware configures the specific mapping of GPIOs to protocols, such as SPI, JTAG, or SWD, based on the loaded configuration. Users can alter the pin distri-

bution by modifying the firmware source code to suit their application requirements.

5.3.1 Protocol ISP – In-System Programming

Compatible with **AVR** microcontrollers, this protocol allows programming and debugging via the SPI interface. The programmer can be used to flash firmware directly into the target device's memory.

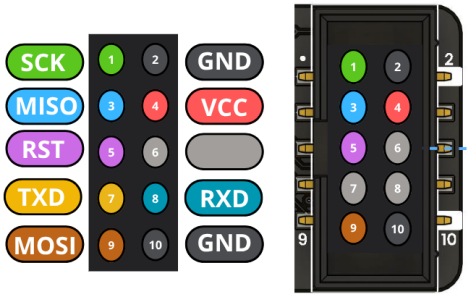


Fig. 5.1: Pinout diagram for CH552 Programmer

Table 5.2: Pinout

PIN	GPIO	I/O
MOSI	1.5	MOSI
MISO	1.6	MISO
CS	3.2	Reset
SCK	1.7	SCK
TXD	3.1	TXD
RXD	3.0	RXD

5.4 Protocol JTAG

Compatible with **CPLD** and **FPGA** devices, this protocol allows programming and debugging via the JTAG interface. The programmer can be used to flash firmware directly into the target device's memory.

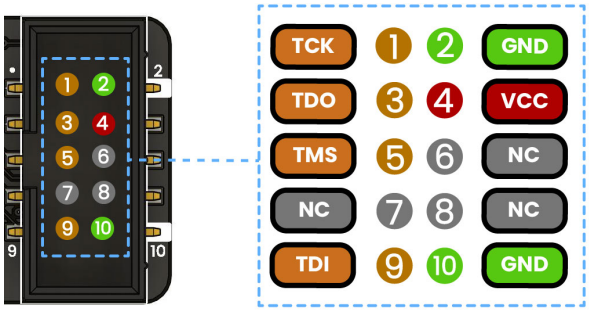


Fig. 5.2: Pinout diagram for CH552 Programmer (JTAG interface)

Table 5.3: Pinout

PIN	GPIO	I/O
TCK	1.7	SCK, TXD1
TMS	3.2	TXD1, INT0, VBUS1, AIN3
TDI	1.5	MOSI, PWM1, TIN3, UCC2, AIN2
TDO	1.6	MISO, RXD1, TIN4

Table 5.4: Pinout NC - Not Connected

PIN	GPIO	I/O
NC 6	3.1	TXD - CH552
NC 7	3.0	RXD - CH552
NC 8	1.4	T2, CAP1, SCS, TIN2, UCC1, AIN1

5.5 Protocol SWD

Compatible with **ARM Cortex-M** microcontrollers, this protocol allows programming and debugging via the SWD interface. The programmer can be used to flash firmware directly into the target device's memory.

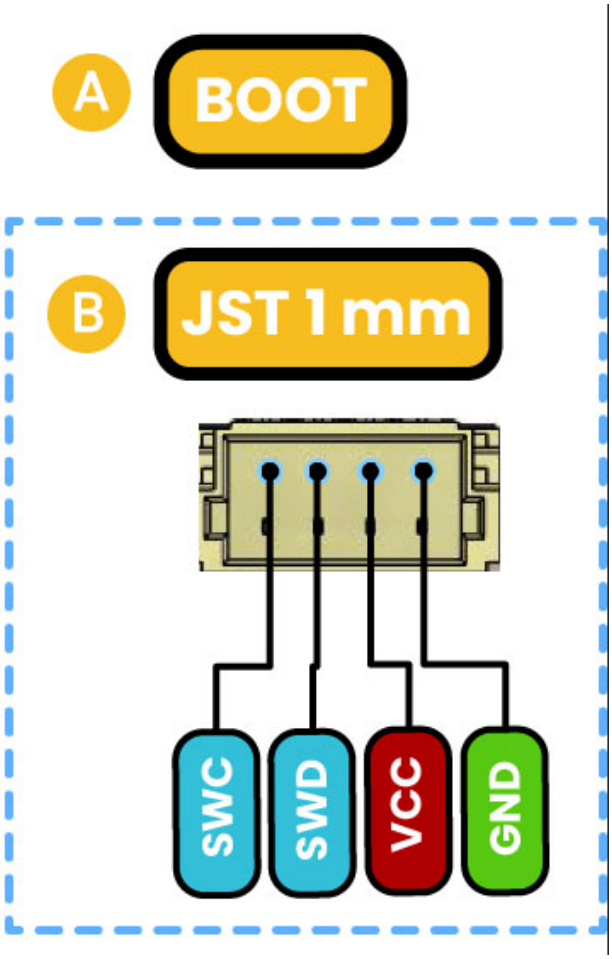


Fig. 5.3: SWD Pinout(JTAG interface)

Table 5.5: Pinout

PIN	GPIO	I/O
SWCLK	1.5	SCK
SWDIO	1.6	MISO

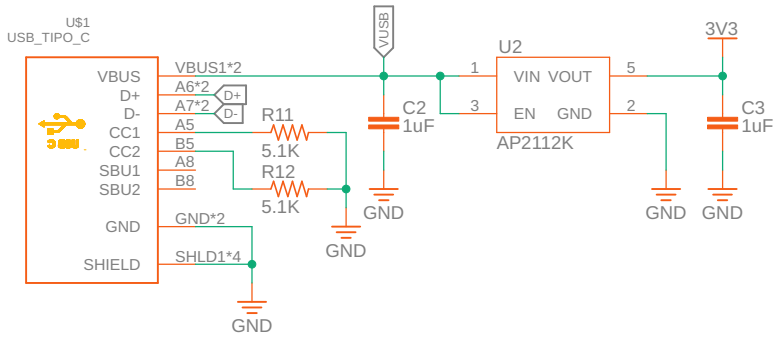
Note: GPIO numbers refer to the CH552 internal ports. Ensure correct firmware pin mapping before connecting external devices.

Table 5.6: Board Reference Table

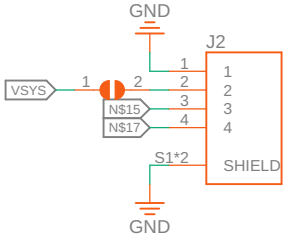
Ref.	Description
IC1	CH552 Microcontroller
U1	AP2112K 3.3V LDO Voltage Regulator
PB1	Boot Push Button
TP1	Reset Test Point
TP2	P3.1 Test Point
L1	Built-In LED
L2	Power On LED
SB1	Solder bridge to enable VCC at JTAG
SB2	Solder bridge to enable VCC at JST
J1	USB Type-C Connector
J2	Low-power I2C JST Connector
J3	JTAG Connector
JP1	Header for SWD or ICSP programming
JP2	Header to Select Operating Voltage Level

APPENDIX A: SCHEMATICS

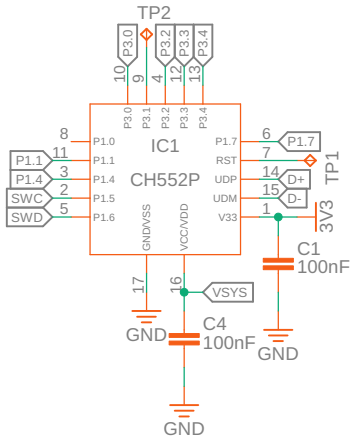
Voltage Regulator



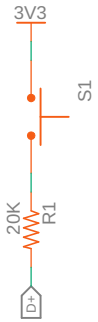
JST



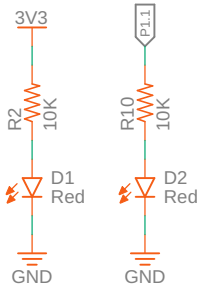
Microcontroller



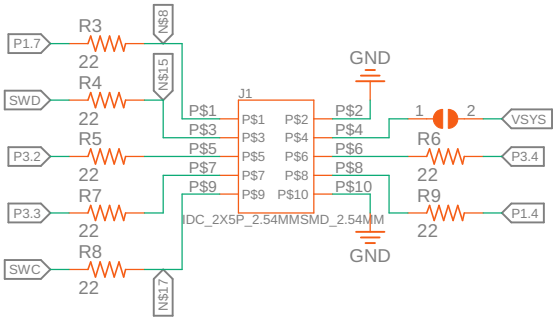
Boot



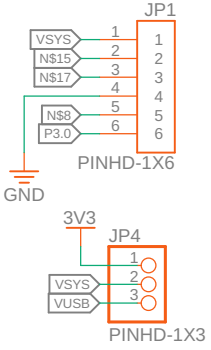
LEDS



IDC Connector



Headers



Mounting Holes

Title: CH552 USB Multi-Protocol Programmer		UNIT ELECTRONICS
SKU: UE0090	REV1.0	
Last date time: 02/05/2025 11:40 a. m.	SHEET: 1 of 1	
File:	Sheet description:	Author:Alberto Villanueva

CHAPTER
SIX

AVR: GETTING STARTED

6.1 Introduction to AVR Microcontrollers

AVR (Advanced Virtual RISC) is a family of microcontrollers originally developed by Atmel, now part of Microchip Technology. Renowned for their simplicity, low power consumption, and ease of use, AVR microcontrollers are widely adopted in embedded systems, including Arduino boards and other DIY electronics projects.

The AVR family spans a variety of models with differing specifications—such as flash memory capacity, I/O pin count, and integrated peripherals. Common examples include the **ATmega** series (e.g., `ATmega328P`, `ATmega2560`) and the **ATtiny** series (e.g., `ATtiny85`, `ATtiny2313`).

6.2 Architecture Overview

AVR microcontrollers are based on a **modified Harvard architecture**, which separates instruction and data memory. This design allows for simultaneous access and contributes to faster instruction execution and efficient memory usage.

Developers typically write code for AVR devices using **AVR Assembly** or higher-level languages like **C/C++**, supported by environments such as **Atmel Studio** or the **Arduino IDE**.

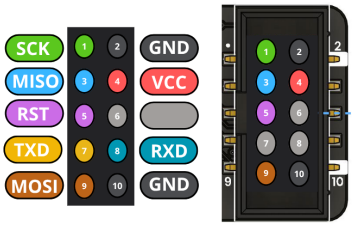


Fig. 6.1: Pinout diagram for CH552 Programmer

6.3 Programming with the CH552 Multi-Protocol Programmer

Table 6.1: Pinout

PIN	GPIO	I/O
MOSI	1.5	MOSI
MISO	1.6	MISO
CS	3.2	Reset
SCK	1.7	SCK
TXD	3.1	TXD
RXD	3.0	RXD

The **CH552 USB Multi-Protocol Programmer** provides robust support for AVR microcontrollers via the **In-System Programming (ISP)** interface. This method enables direct programming of the target microcontroller's **flash memory** and **EEPROM** without removing it from the circuit.

6.3.1 Key Advantages:

- **Non-intrusive**: Program the MCU without desoldering or removing it from the board.
- **Efficient workflow**: Ideal for development, testing, and field updates.
- **Wide compatibility**: Supports common AVR chips used in educational and commercial projects.

6.3.2 Supported Operations:

- Flash memory writing
- EEPROM access
- Fuse and lock bit configuration
- Signature verification

NOTES

CHAPTER
SEVEN

AVR FIRMWARE OVERVIEW

The CH552 Multi-Protocol Programmer relies on the PICO-AVR firmware for correct operation. This firmware, designed to run on CH55x microcontrollers, transforms the CH552 into a versatile USB-to-ISP bridge. It supports ISP AVR programming protocol, making it compatible with a wide range of AVR devices.

The programmer integrates seamlessly with development environments such as the Arduino IDE. It can be selected under Tools → Programmer as either USBasp or a custom driver name, allowing for easy bootloader installation and firmware updates on AVR microcontrollers like the ATmega328P.

Warning: The PICO-AVR firmware is available under the Creative Commons Attribution-ShareAlike 3.0 Unported License.

Additional resources:

- Github: wagiminator
- EasyEDA: wagiminator at EasyEDA
- License details: Creative Commons Attribution-ShareAlike 3.0 Unported License

7.1 Firmware Update Procedure

Note: The recommended implementation workflow is PlatformIO. Use *PlatformIO Support for CH55x* for installation, build, and upload guidance. Legacy Makefile/container builds are documented only in *Optional Docker and spkg*.

To use the **Multi-Protocol Programmer** in PICO ASP mode, build the firmware with the SDK workflow that matches your project. PlatformIO is recommended for new projects. Legacy Makefile examples generate a `.bin` file inside the project **build** directory.

7.1.1 WCHIspStudio Interface

Launch **WCHIspStudio** to upload the firmware.

- In the Object File 1 field, select the path to the generated .bin firmware file.
- Enable the option "Automatic Download When Device Connect".
- Click the … button to browse and confirm the correct firmware path.

Warning: Before connecting the CH552 programmer, **ensure the device is powered with +5V**. Use the onboard switch to select the appropriate voltage.

- Press the Boot button and connect your **Multi-Protocol Programmer** to the computer.
- Await the completion of the firmware update process.

Completion Notice: The firmware has been successfully updated, and the **UNIT CH552 Multi-Protocol Programmer** is now ready for use.

7.1.2 Resources

1. EasyEDA Design Files
2. WCH: CH552 Datasheet
3. SDCC Compiler
4. Blinkinlabs: CH55x SDK for SDCC
5. Thomas Fischl: USBasp
6. Ralph Doncaster: USBasp
7. Deqing Sun: CH55xduino

CHAPTER
EIGHT

AVR: COMPILE AND UPLOAD CODE

8.1 Toolchain Overview

To compile and upload code to an AVR microcontroller, you'll need the **AVR-GCC** toolchain. This includes essential components such as the compiler, assembler, linker, and utilities like `avr-objcopy`.

8.2 Installation

You can download and install the AVR-GCC toolchain from the official Microchip website:

Windows

Download the installer from the Microchip website and follow the installation instructions.

- AVR-GCC Compiler for Microchip Studio

AVR-DUDE is often included with the AVR-GCC toolchain on Windows. If not, you can download it separately:

- AVRDUDE for Windows

Version 8.1 or later is recommended (check compatibility with your architecture).

Linux

You can install the AVR-GCC toolchain using your package manager. For example, on Ubuntu:

```
sudo apt update
sudo apt install gcc-avr binutils-avr avr-
↪libc avrdude
```

Ensure the toolchain is added to your system's `PATH` environment variable for global access.

8.3 Example: Compiling a Blink Program

This example demonstrates how to compile a basic blink program for two common AVR microcontrollers:

- **ATtiny88**
- **ATmega328P**

```
          +-----------------+
PC6 (RST)| 1  ATmega328 28 | PC5 (A5)
PD0 (RX) | 2            27 | PC4 (A4)
PD1 (TX) | 3            26 | PC3 (A3)
     PD2 | 4            25 | PC2 (A2)
PD3 (PWM)| 5            24 | PC1 (A1)
     PD4 | 6            23 | PC0 (A0)
    VCC  | 7            22 | GND
    GND  | 8            21 | AREF
     PB6 | 9            20 | AVCC
     PB7 |10            19 | PB5 (D13)
PD5 (D5) |11            18 | PB4 (D12)
PD6 (D6) |12            17 | PB3 (D11/PWM)
PD7 (D7) |13            16 | PB2 (D10/PWM)
PB0 (D8) |14            15 | PB1 (D9/PWM)
          +-----------------+
```

The program toggles an LED connected to **pin PB0** every second.

8.3.1 Source File

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>

int main(void) {
    DDRB |= (1 << PB0); // Set PB0 as␣
↪output
    while (1) {
        PORTB ^= (1 << PB0); // Toggle PB0
        _delay_ms(1000);
```

(continues on next page)

(continued from previous page)

```
    }
}
```

8.3.2 Compilation Commands

Use the following commands to compile and generate the HEX file:

```
# For ATtiny88
avr-gcc -mmcu=attiny88 -Os -o blink.elf␣
↪blink.c
avr-objcopy -O ihex blink.elf blink.hex

# For ATmega328P
avr-gcc -mmcu=atmega328p -DF_
↪CPU=16000000UL -Os -o blink.elf blink.c
avr-objcopy -O ihex blink.elf blink.hex
```

Explanation of flags:

- `-mmcu=` specifies the target microcontroller.
- `-Os` enables size optimization.
- `-DF_CPU` sets the clock frequency used for timing functions.

8.4 Uploading with AVRDUDE

Once the `.hex` file is generated, you can upload it to the AVR microcontroller using **AVRDUDE**.

8.5 Installation AVRDUDE Linux

You can install AVRDUDE on Linux using your package manager. For example, on Ubuntu:

```
sudo apt-get install avrdude
```

8.5.1 Upload Command

```
avrdude -p m328p -c usbasp -U␣
↪flash:w:blink.hex
```

Explanation:

- `-p m328p` specifies the target device (ATmega328P).
- `-c usbasp` sets the programmer to the Multi-Protocol Programmer.
- `-U flash:w:blink.hex` uploads the hex file to flash memory.

Replace `m328p` with the appropriate identifier for your specific AVR device (e.g., `t88` for ATtiny88). A full list of supported devices is available in the AVRDUDE user manual.

CHAPTER
NINE

AVR: ARDUINO IDE BOOTLOADER

9.1 Installing the Bootloader on ATMEGA328P

This guide explains how to flash the Arduino-compatible bootloader onto an **ATMEGA328** microcontroller using the **UNIT USB Multi-Protocol Programmer**. By following these steps, your ATMEGA328 can function as an **ATMEGA328P**, fully compatible with the Arduino IDE.

9.2 Required Materials

1. **Multi-Protocol Programmer**
2. **ATMEGA328P Microcontroller**

9.3 Hardware Connection

1. Use the FC cable to connect one end to the UNIT Multi-Protocol Programmer and the other to the ICSP interface of the ATMEGA328P.
2. Make sure the **MISO** pin of the programmer is aligned with **pin 1** of the ICSP header on the ATMEGA328P.

9.4 Driver Setup with Zadig

To allow USB communication between your PC and the programmer, install the required drivers using **Zadig**.

9.4.1 Step 1: Identify the COM Port

Connect the programmer to your PC and open the **Device Manager**. Under **Ports (COM & LPT)**, identify the COM port assigned to the device.

9.4.2 Step 2: Open Zadig

Launch Zadig. You should see a window like the following:

Click **Options** → **List All Devices** to display all USB interfaces:

9.4.3 Step 3: Install Drivers

Install the following two drivers:

- **picoASP Interface 0**: Install the **libusbK** driver by clicking **Replace Driver**.
- **SerialUPDI Interface 1**: Install the **USB Serial (CDC)** driver by clicking **Upgrade Driver**.

Once both drivers are installed, your UNIT Multi-Protocol Programmer is ready to flash the bootloader.

9.5 Bootloader Installation Using Arduino IDE

Open the **Arduino IDE** and follow these steps to burn the bootloader onto your ATMEGA328P.

1. **Select the Target Board**

   Navigate to **Tools** → **Board** and choose **ATMEGA328P**.

2. **Choose the Correct Port**

   Under **Tools** → **Port**, select the COM port corresponding to your programmer.

3. **Select the Programmer**

   Under **Tools** → **Programmer**, select the programmer (e.g., **USBasp** or your custom driver name).

4. **Burn the Bootloader**

   Finally, go to **Tools** → **Burn Bootloader**.

Success! Your ATMEGA328P now has a compatible Arduino bootloader installed and is ready for development.

CHAPTER
TEN

ARM CORTEX-M

ARM Cortex-M microcontrollers—such as those found in the STM32, RP2040, PY32, and similar families—are built on ARMv6-M, ARMv7-M, or ARMv8-M architectures. These cores target low-power, high-performance embedded applications.

10.1 Core Families

- Cortex-M0 / M0+ (ARMv6-M): Ultra-low power, suitable for simple tasks.
- Cortex-M3 / M4 / M7 (ARMv7-M): Higher performance; some models (like M4 and M7) support floating-point operations.
- Cortex-M23 / M33 (ARMv8-M): Introduce enhanced security features, such as TrustZone.

10.2 Additional Notes

- The RP2040 uses dual Cortex-M0+ cores (ARMv6-M).
- The PY32 series (e.g., PY32F003) typically uses a Cortex-M0+ core (ARMv6-M).
- The STM32 family spans a wide range—from Cortex-M0 to M7 and even M33 in newer models.

10.2.1 OpenOCD (Open On-Chip Debugger)

OpenOCD is an open-source software tool that facilitates debugging, in-system programming, and boundary-scan testing of embedded devices. It supports a diverse range of microcontrollers including STM32, RP2040, and PY32. Through interfaces such as JTAG or SWD, OpenOCD provides a command-line interface for effective interaction with the target devices.

10.2.2 PyOCD: A Python Interface to OpenOCD

PyOCD simplifies the use of OpenOCD by providing a Python-based high-level interface. It offers a Python API that abstracts the complexities of the OpenOCD command-line interface, making it easier to program and debug microcontrollers. PyOCD is user-friendly, flexible, and designed to be extensible for various microcontroller interactions.

Tool	Functionality
OpenOCD	Provides a command-line interface for debugging and programming microcontrollers.
PyOCD	Offers a Python API for seamless interaction with microcontrollers via OpenOCD.

10.3 ARM Cortex-M Debug Capabilities

10.3.1 Debugging Interface of ARM Cortex-M

The ARM Cortex-M architecture includes a comprehensive debug interface that allows external tools to access the microcontroller core for debugging and programming tasks. This interface includes a suite of registers and commands that empower debuggers to halt the core, access memory, and manage the execution of programs.

10.3.2 Connection Through Debug Pins

Access to the debug interface is facilitated through specific pins on the microcontroller, such as SWDIO, SWCLK, and nRESET. These are linked to a debug adapter like the ST-Link or CMSIS-DAP, which forms the physical bridge between the host computer and the target device.

10.4 CMSIS-DAP: A Standardized Debug Adapter

10.4.1 Overview of CMSIS-DAP

CMSIS-DAP (Cortex Microcontroller Software Interface Standard - Debug Access Port) serves as a standardized debug adapter for ARM Cortex-M devices. It is endorsed by OpenOCD and provides a cost-effective solution compared to proprietary debug adapters.

10.5 SWD Communication Protocols

10.5.1 Signals and Operations

The SWDIO (Serial Wire Debug Input/Output) and SWCLK (Serial Wire Clock) signals are integral for communication over the SWD interface. SWDIO is a bidirectional line used for transmitting debug commands and data, while SWCLK is a clock signal that ensures synchronized data transfer between the host and the target.

10.5.2 Advantages of SWD Over JTAG

Utilizing a two-wire connection, the SWD interface minimizes the pin count required compared to the traditional JTAG interface. This reduction is particularly beneficial for microcontrollers where pin availability is limited.

CHAPTER
ELEVEN

USING OPENOCD

To program your microcontroller using OpenOCD, use the following command:

```
openocd -f interface/cmsis-dap.cfg -f␣
↪target/rp2040.cfg \
        -c "init" -c "reset init" \
        -c "flash write_image erase blink.
↪bin 0x10000000" \
        -c "reset run" -c "shutdown"
```

11.1 Supported Microcontrollers

This guide supports the following microcontrollers:

Micro-controller	Description
RP2040	Low-cost microcontroller with a dual-core ARM Cortex-M0+ processor.
STM32F1	Budget-friendly microcontroller with an ARM Cortex-M3 processor.
STM32F4	Cost-effective microcontroller with an ARM Cortex-M4 processor.

11.1.1 Selecting Your Microcontroller Target

Each microcontroller requires a specific configuration file. This file is a script that instructs OpenOCD on how to communicate with the microcontroller. The configuration file is tailored to the microcontroller and the debugger interface.

Microcon-troller	Configuration File
RP2040	`rp2040.cfg`
STM32F103C8	`stm32f103c8t6.cfg`
STM32F411	`stm32f411.cfg`

CHAPTER
TWELVE

PYOCD

PyOCD is a command-line tool for interacting with ARM Cortex-M microcontrollers. It supports flashing, erasing, debugging, and low-level memory access using CMSIS-DAP, DAPLink, ST-Link, and other probes.

12.1 Basic Syntax

```
pyocd <command> [OPTIONS]
```

You can specify the target using `-t <target>` or through a configuration file using `--config <file.yaml>`.

12.2 Configuration File

Use a YAML configuration file to define global options:

```yaml
# Misc/pyocd.yaml
target_override: py32f003x6
frequency: 1000000
log_level: info
```

To load this config:

```
pyocd erase --config ./Misc/pyocd.yaml
```

12.3 Commands

12.3.1 erase

Erase flash memory of the target device.

```
pyocd erase -t py32f003x6 [OPTIONS]
```

Options:

- `--chip`: Erase the entire flash memory.
- `--sector <ADDR>`: Erase a single sector by address.
- `--config <file.yaml>`: Load configuration from YAML file.

Example:

```
pyocd erase -t py32f003x6 --chip --config .
↪/Misc/pyocd.yaml
```

12.3.2 flash

Flash a binary, hex, or ELF file to the target.

```
pyocd flash <file> -t py32f003x6 [OPTIONS]
```

Options:

- `--base-address <addr>`: Override the base address (for .bin files).
- `--verify`: Verify flash contents after writing.
- `--erase=chip|sector|auto`: Select erase mode.
- `--format bin|hex|elf`: Force file format.
- `--config <file.yaml>`: Load YAML config.

Example:

```
pyocd flash firmware.hex -t py32f003x6 --
↪erase=chip --verify
```

12.3.3 gdbserver

Start a GDB server for remote debugging.

```
pyocd gdbserver -t py32f003x6 [OPTIONS]
```

Options:

- `--port <number>`: GDB server port.
- `--telnet-port <number>`: Telnet monitor port.
- `--persist`: Keep the server alive after GDB disconnects.
- `--config <file.yaml>`: Load YAML config.

12.3.4 reset

Reset the target microcontroller.

```
pyocd reset -t py32f003x6 [OPTIONS]
```

Options:

- `--halt`: Halt the CPU after reset.
- `--config <file.yaml>`: Load YAML config.

12.3.5 list

List connected debug probes and supported targets.

```
pyocd list
```

12.3.6 read / write

Read and write memory directly.

```
pyocd read32 0x08000000 4
pyocd write32 0x20000000 0x12345678
```

12.4 CMSIS-Pack Targets

To add custom target support (e.g., `py32f003x6`), use:

```
pyocd pack install py32f003x6
```

Or add a local *.pdsc* file:

```
pyocd pack add ./path/to/PY32F003.pdsc
```

12.5 Example Workflow

```
pyocd list
pyocd erase -t py32f003x6 --chip
pyocd flash build/main.hex -t py32f003x6 --
↪verify
pyocd gdbserver -t py32f003x6
pyocd reset -t py32f003x6 --halt
```

12.6 References

- https://pyocd.io/
- https://github.com/pyocd/pyocd

12.7 Command Help

```
pyocd --help
pyocd flash --help
pyocd erase --help
```

CHAPTER
THIRTEEN

RP2040: AN INTRODUCTION

The RP2040 is an efficient and cost-effective microcontroller developed by Raspberry Pi. It features a dual-core ARM Cortex-M0+ processor, flexible I/O capabilities, and a wide range of peripherals, making it suitable for both hobbyist and professional applications.

Note: This documentation is continuously updated and incorporates key concepts from the official Raspberry Pi Pico SDK documentation.

Tip: A Linux operating system is recommended, as the Pico-SDK is primarily developed and tested on Linux. Windows may require additional setup. macOS has not been tested in this documentation process.

This guide provides a comprehensive introduction to working with RP2040 microcontrollers. It covers setting up the development environment, installing the required software, and writing code for RP2040-based projects.

13.1 Multi-Protocol Programmer for RP2040

The Multi-Protocol Programmer is a versatile tool for programming and debugging RP2040 microcontrollers. It enables straightforward and reliable programming using the SWD (Serial Wire Debug) interface.

13.2 Programming RP2040 with the Multi-Protocol Programmer

The Multi-Protocol Programmer supports programming of RP2040 microcontrollers via the SWD interface. This interface allows direct access to the target microcontroller's flash memory and supports debugging functionality. The programmer is compatible with RP2040f10x and RP2040f4xx series.

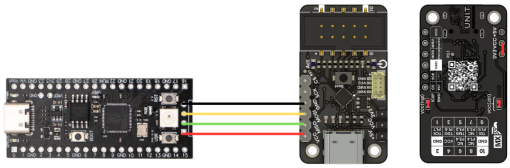


Fig. 13.1: Pinout diagram for RP2040

13.3 DualMCU RP2040 Programming with Multi-Protocol Programmer

The Multi-Protocol Programmer uses the SWD interface to program RP2040 microcontrollers. Follow these steps to program your RP2040 device:

1. **Connect the Multi-Protocol Programmer** to your computer via USB.
2. **Open Visual Studio Code** or another editor of your preference.
3. **Install the required extensions** for RP2040 development, such as "C/C++" and "CMake Tools".
4. **Create a new project** or open an existing one.
5. **Write your code** in C or C++ using the RP2040 SDK.
6. **Configure the build system** to use the RP2040 SDK and integrate the CH552 Multi-Protocol Programmer.
7. **Build the project** to generate the binary firmware file.

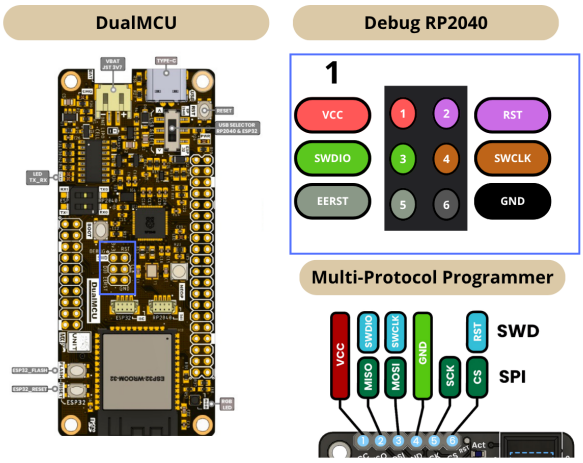


Fig. 13.2: DualMCU RP2040 Connection

CHAPTER
FOURTEEN

RP2040 FIRMWARE

14.1 Firmware update

Note: The recommended implementation workflow is PlatformIO. Use *PlatformIO Support for CH55x* for installation, build, and upload guidance. Legacy Makefile/container builds are documented only in *Optional Docker and spkg*.

To start using your **Multi-Protocol Programmer** as a PICO DAP, build the firmware with the SDK workflow that matches your project. PlatformIO is recommended for new projects. Legacy Makefile examples create a `.bin` file inside the **build** folder.

14.1.1 WCHIspStudio

Then open **WCHIspStudio** to upload the firmware

- In Object File 1 make sure to enter the correct path to the directory with the firmware.
- Make sure the "Automatic Download When Device Connect" option is enabled.
- To add the path you have to click on this bottom "…" and check.

Warning: Before connecting your Multi-Protocol Programmer, make sure to power it with +5V. You can do this by setting your switch to +5V.

- Push the Boot bottom and connect your **Multi-Protocol Programmer** to your computer.
- Wait until de firmware has finished updating your device.

Done! Now you can use your UNIT CH552 Multi-Protocol Programmer!

14.2 Create a new project in Pico SDK

14.2.1 Installation

Open Visual Studio Code, in the extensions menu, search Raspeberry Pi Pico:

And install the official version from Raspberry:

In the Activity Bar, you will find the icon of your new extension in Visual Studio Code.

In the general menu, click on "New C/C++ Project"

Once the project is created, you will need to configure it. For this example, we will use a Raspberry Pico, UART, SPI and Console over UART and USB

14.2.2 Project

Inside the generated project, you will find these files.

Open the .c file, here we can change or modify the source code.

14.2.3 Examples

Here are some examples for a Raspeberry Pi Pico:

1. Hello, World! Open a serial monitor and see what's happening!

```
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/uart.h"

// UART defines
// By default the stdout UART is uart0, so␣
↪we will use the second one
#define UART_ID uart1
#define BAUD_RATE 115200

// Use pins 4 and 5 for UART1
```

(continues on next page)

(continued from previous page)

```
// Pins can be changed, see the GPIO␣
↪function select table in the datasheet␣
↪for information on GPIO assignments
#define UART_TX_PIN 4
#define UART_RX_PIN 5



int main()
{
    stdio_init_all();

    // Set up our UART
    uart_init(UART_ID, BAUD_RATE);
    // Set the TX and RX pins by using the␣
↪function select on the GPIO
    // Set datasheet for more information␣
↪on function select
    gpio_set_function(UART_TX_PIN, GPIO_
↪FUNC_UART);
    gpio_set_function(UART_RX_PIN, GPIO_
↪FUNC_UART);

    // Use some the various UART functions␣
↪to send out data
    // In a default system, printf will␣
↪also output via the default UART

    // Send out a string, with CR/LF␣
↪conversions
    uart_puts(UART_ID, " Hello, UART!\n");

    // For more examples of UART use see␣
↪https://github.com/raspberrypi/pico-
↪examples/tree/master/uart

    while (true) {
        printf("Hello, World!\n");
        sleep_ms(1000);
    }
}
```

2. Blink

```
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/uart.h"

// UART configuration
#define UART_ID uart1
#define BAUD_RATE 115200
#define UART_TX_PIN 4
```

(continues on next page)

(continued from previous page)

```
#define UART_RX_PIN 5

// On-board LED pin (GPIO 25)
#define LED_PIN 25

int main()
{
    // Initialize standard I/O (required␣
↪for printf to work)
    stdio_init_all();

    // Initialize UART1 with the specified␣
↪baud rate
    uart_init(UART_ID, BAUD_RATE);

    // Configure UART TX and RX GPIO␣
↪functions
    gpio_set_function(UART_TX_PIN, GPIO_
↪FUNC_UART);
    gpio_set_function(UART_RX_PIN, GPIO_
↪FUNC_UART);

    // Initialize GPIO 25 for LED and set␣
↪it as output
    gpio_init(LED_PIN);
    gpio_set_dir(LED_PIN, GPIO_OUT);

    // Send a welcome message through UART1
    uart_puts(UART_ID, " Hello, UART!\n");

    while (true) {
        // Turn on the LED
        gpio_put(LED_PIN, 1);
        uart_puts(UART_ID, "LED ON\n");  //
↪ Send status via UART
        sleep_ms(500);                   //
↪ Wait 500 milliseconds

        // Turn off the LED
        gpio_put(LED_PIN, 0);
        uart_puts(UART_ID, "LED OFF\n"); //
↪ Send status via UART
        sleep_ms(500);                   //
↪ Wait another 500 milliseconds
    }
}
```

14.2.4 Flashing

- Once you have your example ready to upload, just click on the Pico SDK icon on the Activity Bar.
- Select the "Flash Project (SWD)" option.

Note: To program a Raspberry Pico, use the SWD protocol. For more information, check the pinout.

Warning: The Raspberry Pi Pico operates at 3.3V. Switch to 3.3V before connecting your device.

CHAPTER
FIFTEEN

STM32: GETTING STARTED

STMicontrollers is a family of 32-bit microcontrollers designed and manufactured by STMicroelectronics. They are widely used in embedded systems and IoT applications due to their low power consumption, high performance, and rich peripheral set. STM32 microcontrollers are based on the ARM Cortex-M architecture and are available in a wide range of series, each tailored to specific applications and requirements.

15.1 Getting Started with STM32

This documentation provides a comprehensive guide to getting started with STM32 microcontrollers. It includes information on setting up the development environment, installing the necessary software, and writing code for STM32 microcontrollers.

This technical documentation is a work in progress and is an adaptation of documentation from the STM32CubeIDE and STM32CubeMX software tools. It uses the ARM-GCC compiler and visual studio code as the IDE.

CH552 Multi-Protocol Programmer is a versatile tool that supports programming and debugging of STM32 microcontrollers. It provides a simple and efficient way to program STM32 devices using the SWD (Serial Wire Debug) interface.

15.2 Programming STM32 with CH552 Multi-Protocol Programmer

Currently, the CH552 Multi-Protocol Programmer supports programming STM32 microcontrollers using the SWD interface. This allows for direct programming of the target microcontroller's flash memory and debugging capabilities STM32f10x series and STM32f4xx series.

CHAPTER
SIXTEEN

STM32 FIRMWARE

16.1 Firmware update

Note: The recommended implementation workflow is PlatformIO. Use *PlatformIO Support for CH55x* for installation, build, and upload guidance. Legacy Makefile/container builds are documented only in *Optional Docker and spkg*.

To start using your **Multi-Protocol Programmer** as a PICO DAP, build the firmware with the SDK workflow that matches your project. PlatformIO is recommended for new projects. Legacy Makefile examples create a `.bin` file inside the **build** folder.

16.1.1 WCHIspStudio

Then open **WCHIspStudio** to upload the firmware

- In Object File 1 make sure to enter the correct path to the directory with the firmware.
- Make sure the "Automatic Download When Device Connect" option is enabled.
- To add the path you have to click on this bottom "…" and check.

Warning: Before connecting your Multi-Protocol Programmer, make sure to power it with +5V. You can do this by setting your switch to +5V.

- Push the Boot bottom and connect your **Multi-Protocol Programmer** to your computer.
- Wait until de firmware has finished updating your device.

Done! Now you can use your UNIT CH552 Multi-Protocol Programmer!

16.2 Create a new project in PlatformIO

16.2.1 Installation

Open Visual Studio Code, in the extensions menu, search PlatformIO:

And install the official version from PlatformIO:

In the Activity Bar, you will find the icon of your new extension in Visual Studio Code.

In the general menu, click on "Create New Project"

In the Quick Access menu, we can open an existing file for our STM32F4xx or create a new one:

Next, in the Project Wizard:

- We will name our file.
- Select the specific model of our STM32F4xx board.
- Choose the framework (for this example, we will use CMSIS)
- Specify the location where we want to save the project.

16.2.2 Project

Inside the generated project, you will find this project structure.

If you need to change your COM port, follow these steps:

Inside the .ini file, make sure you have this configuration:

```
[env:blackpill_f411ce]
platform = ststm32
board = blackpill_f411ce
framework = cmsis
upload_protocol = cmsis-dap
debug_tool = cmsis-dap
```

Note: If you are using a different board, change the "[env: ...]" and "board = ..." to match your board model.

Here is a list of common STM32 microcontrollers and their corresponding **board** identifiers used in PlatformIO's **platformio.ini** file.

Table 16.1: STM32 Microcontrollers

Microcontroller / Board	PlatformIO board name
STM32F103C8 (Blue Pill)	`bluepill_f103c8`
STM32F401RE (Nucleo)	`nucleo_f401re`
STM32F446RE (Nucleo)	`nucleo_f446re`
STM32F103RB	`genericSTM32F103RB`
STM32F407VG (Discovery)	`disco_f407vg`
STM32F411CEU6 (Blackpill)	`blackpill_f411ce`

16.2.3 Example

The following example is a Blink using GPIOC (PC13) as an output.

```
#include "stm32f4xx.h"

// Simple software delay loop
void delay(volatile uint32_t t) {
    while (t--);
}

int main(void) {
    // 1. Enable the clock for GPIOC␣
↪(required before accessing GPIOC␣
↪registers)
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOCEN;

    // 2. Configure pin PC13 as general-
↪purpose output (MODER13 = 0b01)
    GPIOC->MODER &= ~(0x3 << (13 * 2)); //␣
↪Clear MODER13 bits
    GPIOC->MODER |=  (0x1 << (13 * 2)); //␣
↪Set MODER13 to output mode

    // 3. Main loop
    while (1) {
        GPIOC->ODR ^= (1 << 13); // Toggle␣
↪PC13 (invert LED state)
        delay(500000);           // Simple␣
```

(continues on next page)

(continued from previous page)

```
↪delay (not accurate, for testing␣
↪purposes)
    }
}
```

16.2.4 Flashing

- Once you have your example ready to upload, just click on the PlatformIO icon on the Activity Bar.
- Select the "Upload" option.

Note: To program a STM32F4xx, use the SWD protocol. For more information, check the pinout.

Warning: The STM32F4xx operates at 3.3V. Switch to 3.3V before connecting your device.

CHAPTER
SEVENTEEN

CPLD/FPGA

This firmware is not an official programmer from Altera or Intel. Instead, it is a JTAG programmer for CPLD/FPGA devices based on the WCH CH552 USB microcontroller. It supports the Intel (formerly Altera) MAX II series and is compatible with Quartus Prime Lite.

Warning: This firmware is a third-party implementation and **is not affiliated with or endorsed by Altera or Intel**. It is intended for educational, development, prototyping, and other lawful purposes only.

Ensure that you comply with all applicable laws and regulations when using this firmware. **Use is at your own risk.** The user assumes all responsibility for any consequences, including potential legal implications. The author and distributor of this firmware **are not liable** for any damage, misuse, or legal issues arising from its use.

Proceed with caution and discretion.

For programming Altera FPGA and CPLD devices, the Quartus software is used. A free version, Quartus Lite, is available. Quartus is the suite from Altera (now Intel) for programming, configuring, and using its products, and it fully supports the USB Blaster programmer and all its features.

17.1 Quick installation

Note: To download the Intel Quartus Prime software, visit Quartus Prime Installer.

1. **Open the file .exe**
    a. In the 24.1 version, add your devices
    b. Check the option "Agree to Intel License Agreement"
    c. Click on "Download and Install"
    d. Wait until the installation has completed succesfully

17.2 Create a new project in Quartus

Once installed the software and the driver for your CH552, we can create a new project. It's necessary that you've conected your device to your computer.

In the upper left, you will find the File > New Project Wizard tab.

a. You will see the New Project Wizard window
a. Select the option "Next".
b. Choose an appropriate name to start the project.
c. Select the folder where you will save your project.
d. In the project type, select the option empty project as shown above:
e. Afterwards, click Next.
f. For this example, it is not necessary to add additional files.
g. Afterwards, click Next.
h. This is the where you can select your device, package, pin count, core speed grade.
i. EDA Tools Settings.

Note: These options do not define the device family (e.g., MAX II, Cyclone IV, etc.); rather, they specify how Quartus will connect to third-party tools (ModelSim, Synopsys, etc.). If you do not have any external EDA tools, you may leave all options set to <None>, and Quartus will use its internal functionality to compile and generate programming files.

j. In this window, you will be able to see a summary of your settings.

k. Afterwards, click Finish.

CHAPTER
EIGHTEEN

CPLD FIRMWARE

18.1 Firmware update

Warning: This firmware is a third-party implementation and **is not affiliated with or endorsed by Altera or Intel**. It is intended for educational, development, prototyping, and other lawful purposes only.

Ensure that you comply with all applicable laws and regulations when using this firmware. **Use is at your own risk.** The user assumes all responsibility for any consequences, including potential legal implications. The author and distributor of this firmware **are not liable** for any damage, misuse, or legal issues arising from its use.

Proceed with caution and discretion.

Note: The recommended implementation workflow is PlatformIO. Use *PlatformIO Support for CH55x* for installation, build, and upload guidance. Legacy Makefile/container builds are documented only in *Optional Docker and spkg*.

To start using your **Multi-Protocol Programmer** as a CPLD/FPGA programmer, follow these steps:

```
cd examples/USB/Prog/CPLD/src
```

In that directory, open **descriptor.h** inside that file, change lines 13 and 14 of the code as shown below:

Only change the **zeros** for **ones**

Once you have changed this data, build the firmware with the SDK workflow that matches your project. PlatformIO is recommended for new projects. Legacy Makefile examples create a `.bin` file inside the **build** folder. Then open **WCHIspStudio** to upload the firmware

- In Object File 1 make sure to enter the correct path to the directory with the firmware.
- Make sure the "Automatic Download When Device Connect" option is enabled.
- To add the path you have to click on this bottom "..." and check.

Note: Before connecting your Multi-Protocol Programmer, make sure to power it with +5V. You can do this by setting your switch to +5V.

- Push the Boot bottom and connect your **Multi-Protocol Programmer** to your computer.
- Wait until de firmware has finished updating your device.

Done! Now you can use your UNIT CH552 Multi-Protocol Programmer!

Note: To program an FPGA and a CPLD, use the JTAG Protocol. For more information, check the pinout.

CHAPTER
NINETEEN

HOW TO GENERATE AN ERROR REPORT

This guide explains how to generate an error report using GitHub repositories.

19.1 Steps to Create an Error Report

1. **Access the GitHub Repository**
   Navigate to the GitHub repository where the project is hosted.
2. **Open the Issues Tab**
   Click on the "Issues" tab located in the repository menu.
3. **Create a New Issue**
   - Click the "New Issue" button.
   - Provide a clear and concise title for the issue.
   - Add a detailed description, including relevant information such as:
     - Steps to reproduce the error.
     - Expected and actual results.
     - Any related logs, screenshots, or files.
4. **Submit the Issue**
   Once the form is complete, click the "Submit" button.

19.2 Review and Follow-Up

The development team or maintainers will review the issue and take appropriate action to address it.